

May 26, 2025

o3/o4-mini Function Calling Guide

This recipe is archived and may reference outdated models or APIs.



Bill Chen (OpenAI) , Prashant Mital (OpenAI)

[View on GitHub](#)[Download raw](#)

Introduction

The o3/o4-mini models are the latest in our o-series of models trained to think for longer before responding. They are the smartest models we've released to date and represent a significant step forward from o1/o3-mini in tool calling capabilities. These models are trained to use tools natively within their chain of thought (CoT) which unlocks improved reasoning capabilities around when and how to use tools. We've released a guide on how to [call functions](#) with these models via the responses API, this guide builds on top of that and tells you how you can get the best function calling performance with these models.

Prompt guidance for better function calling performance

To fully utilize function calling intelligence behind o3/o4-mini models, we recommend a few best practices in both developer prompts and function descriptions.

A quick note on developer prompt, system prompt, and function descriptions for reasoning models

We introduced developer messages to make it explicit to reasoning models that an instruction is coming from the developer. In o-series models, any system message provided by the developer is automatically converted to a developer message internally. For practical purposes, you can treat the developer prompt as analogous to the traditional system prompt—but for clarity and correctness, this guide refers to all such instructions as developer prompts/messages.

[Ask AI](#)

Loading docs agent...

1. General context: In line with general prompt engineering best practices, role prompting is helpful in setting the base behavior, tone and outlining the set of actions that are possible. For example:

You are an AI retail agent.



As a retail agent, you can help users cancel or modify pending orders, return or exchange delivered orders, modify their default user address, or provide information about their own profile, orders, and related products.

2. Function Call ordering: o3/o4-mini are trained to accomplish goals with tools. However, it can make mistakes in the order of the tool calls. To guard against these cases, it is recommended to explicitly outline the orders to accomplish certain tasks. For example, to guard against the failure case that a coding agent

Loading docs agent...

- Use tools when:
 - The user wants to cancel or modify an order.
 - The user wants to return or exchange a delivered product.
 - The user wants to update their address or contact details.
 - The user asks for current or personalized order or profile info.
- Do not use tools when:
 - The user asks a general question like "What's your return policy?"
 - The user asks something outside your retail role (e.g., "Write a poem").

If a task is not possible due to real constraints (For example, trying to cancel an already delivered order), explain why clearly and do not call tools blindly.

Loading docs agent...

storage is needed and the file does not already exist.

- Only call this function if the target directory exists. Check first using the ``directory_check`` tool.
- Do not use for temporary or one-off content—prefer direct responses for those cases.
- Do not overwrite existing files. Always ensure the file name is unique.
- Do not overwrite existing files.
 - If replacement is intended and confirmed, use ``file_delete`` followed by ``file_create``, or use ``file_update`` instead.

2. Few shot prompting: While reasoning models do not benefit from few-shot prompting as much as non-reasoning models, we found that few shot prompting can improve tool calling performance, especially when the model struggles to accurately construct function arguments. For example, here is an example tool description for a grep tool passed in as tool description:

Loading docs agent...

hits.

You can fine-tune the search by specifying inclusion or exclusion rules based on file types or path patterns.

This method is ideal when searching for literal text snippets or specific regular expressions.

It offers more accuracy than semantic methods when the goal is to locate a known string or structure.

It's generally recommended over semantic search when you're looking for a specific identifier—such as a function name, variable, or keyword—within a defined set of directories or file types.

This performs poorly because much of the prompt is not prescriptive and the most important rules for how to construct the argument are not front and center. The previous prompt scored 6% higher on a tool calling accuracy eval for using this ripgrep tool compared to the one above.

Loading docs agent...

from o3, such as stating it does not have enough time to complete a task, promising to follow up separately, or giving terse answers even when explicitly prompted to provide more detail. We have found that the following steps help ameliorate this behavior:

a. Start a new conversation for unrelated topics:

When switching to a new or unrelated topic, begin a fresh conversation thread rather than continuing in the same context. This helps the model focus on the current subject and prevents it from being influenced by previous, irrelevant context, which can sometimes lead to incomplete or lazy responses. For example, if you were previously discussing code debugging and now want to ask about documentation best practices, which does not require previous conversation context, start a new conversation to ensure clarity and focus.

Loading docs agent...

Responses API

Reasoning Items for Better Performance

We've released a [cookbook](#) detailing the benefits of using the responses API. It is worth restating a few of the main points in this guide as well. o3/o4-mini are both trained with its internal reasoning persisted between tool calls within a single turn. Persisting these reasoning items between tool calls during inference will therefore lead to higher intelligence and performance in the form of better decision in when and how a tool gets called. Responses allow you to persist these reasoning items (maintained either by us or yourself through encrypted content if you do not want us to handle state-management) while Chat Completion doesn't. Switching to the responses API and allowing the model access to reasoning items between function calls is the easiest way to squeeze out as much performance as possible for function calls. Here is an the example in the cookbook, reproduced for convenience, showing

Loading docs agent...

```
25     },
26     "strict": True
27 }]
28
29 context = [{"role": "user", "content": "What's the weather
like in Paris today?"}]
30
31 response = client.responses.create(
32     model="o3",
33     input=context,
34     tools=tools,
35     store=False,
36     include=["reasoning.encrypted_content"] # Encrypted
chain of thought is passed back in the response
37 )
38
39
```

Loading docs agent...

tools and features in the responses API which makes it easier to build agentic applications that connect to external services; With built-in tools in the Responses API, developers can create more capable agents with a single API call.

You can mix and match hosted tools and custom tools in the same session. This unlocks powerful composition patterns, but it also makes tool routing clarity critical. Here are a couple of concrete recommendations:

1. Explicitly define tool usage boundaries in the developer prompt: If multiple tools can fulfill similar roles (e.g. both the python tool and a custom calculator), instruct the model which tool is preferred and when. This reduces ambiguity, improves accuracy, and avoids tool overuse or underuse. :

You are a helpful research assistant with access to the following tools:



Loading docs agent...

Use ``python`` for general math, data parsing, unit conversion, or logic tasks that can be solved without external lookup—for example, computing the total cost from a list of prices.



Use ``calculate_shipping_cost`` when the user asks for shipping estimates, as it applies business-specific logic and access to live rate tables. Do not attempt to estimate these using the ``python`` tool.

When both could be used (e.g., calculating a delivery fee), prefer ``calculate_shipping_cost`` for accuracy and policy compliance. Fall back to ``python`` only if the custom tool is unavailable or fails.

Loading docs agent...

another using the overall developer prompt. [Here](#) is a great example from the MCP tool guide.

Frequented Asked Questions (FAQ)

Q: How many functions is too many?

A: For o3 and o4-mini models, there is no hard upper limit on the number of functions, but practical guidance does exist based on both training data distribution and observed model behavior. As of May 2025, any setup with fewer than ~100 tools and fewer than ~20 arguments per tool is considered in-distribution and should perform within expected reliability bounds. Performance still depends on your prompt design and task complexity.

Even if you are technically within training distribution, more tools can introduce ambiguity or confusion. Here are key considerations:

Loading docs agent...

internal parsing and structuring, which can help prevent issues like partially filled nested objects or invalid field combinations. With deeply nested objects, especially ones with repeated or semantically similar field names, the model is more likely to omit or misuse arguments.

Nesting can help organize complex logic, but needs additional care: For domains that naturally involve structured input, like configuration payloads, rich search filters, or form submissions, nesting helps organize related parameters. However, you must use techniques like clear field descriptions, `anyOf` logic, or strict schemas to guard against invalid argument combinations and improve model reliability

The best way to choose is to test with your own evals and measure success. There's no "one-size-fits-all" because invocation behaviors are emergent and prompt-sensitive

Loading docs agent...